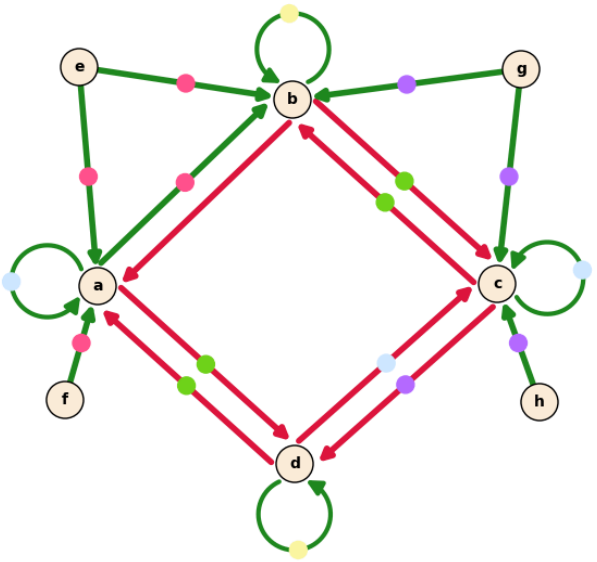
2025 - 2026

M1 GENIOMHE-AI

Internship Report: Graphical Design of Boolean Networks - Platform BooN



First Name: Boomika
Last Name: SELVARADJOU

Organism: Laboratory IBISC - Evry
Supervisor: Pr. Franck DELAPLACE
University Tutor: Pr. Christophe AMBROISE
& Pr. Valérie CHAUDRU.

❖ Acknowledgments:

Upon the completion of this report, I would like to express my sincere gratitude to my supervisor, teachers, administration personnel, and all those who provided guidance and support throughout the research and writing process.

Boomika SELVARADJOU
University Evry Val d'Essonne - Paris Saclay
June 22, 2026

❖ Summary:

This internship project focuses on improving the graphical user interface of the open-source platform BooN, which is used for the modelling and analysis of Boolean networks in biological systems.

Boolean networks are widely used in biology to model gene regulatory networks, signaling pathways and cellular mechanisms, involving interactions between genes and proteins. These approaches have applications in cancer research, like helping the study of disease mechanisms through modelling of Boolean networks and identifying potential therapeutic drug targets.

The project aims to redesign and enhance the current version of BooN graphical user interface to simplify the creation and manipulation of Boolean networks for users. We want to implement the graphical construction of networks and encoding of the Boolean networks into DNF formulas. These enhancements will facilitate modelling and support research on cancer mechanisms and target drug discovery.

The implementation relies on the Python PyQt5 framework and QtDesigner for graphical interface development. The analytical methods integrated into BooN includes computations such as network dynamics computation, stable state and controllability analysis.

Altogether, the project aims to develop a more efficient and user-friendly version of the software, to make Boolean network analysis more accessible to researchers and users.

Key Words: Boolean Networks; Graphical User Interface; Systems Biology; Computational Biology; PyQt5.

❖ Table of Contents:

I. Introduction.....5
A. Scientific Context.....5
B. Objective of internship.....7
II. Material & Methods.....8
A. Description of Project.....8
B. Methods and tools used.....11
III. Results obtained.....15
IV. Discussion & Perspectives.....21
V. Conclusion.....22
VI. Bibliography.....23
VII. Appendices.....24

❖ Glossary:

Abbreviation:	Definition:
Bax	Bcl-2-associated X protein; A pro-apoptotic protein.
BoolNet	Software package for Boolean network analysis.
BooN	Open-source software platform for Boolean network modelling, computing and analysis.
CycD1	Cyclin D1; A cell-regulatory protein that helps transition from G1 phase to S phase of the cell cycle.
DNF	Disjunctive Normal Form
Git	Distributed version control system used to track modifications in source code, enabling version management and reproducibility.
GUI	Graphical User Interface
ODE	Ordinary Differential Equations
PyQt5	Python bindings for Qt Framework Version 5
SAT	Boolean Satisfiability
SBML	Systems Biology Markup Language
XML	eXtensible Markup Language

I. Introduction

A. Scientific Context

In biology, living organisms rely on complex biological mechanisms that involve multiple genes, proteins and signalling pathways that interact through interconnected regulatory networks. These interactions control essential cellular processes, such as proliferation, differentiation, apoptosis, metabolism and immune responses. Their study is crucial in modern biological research, to understand more about the mechanisms involved in normal cellular behaviour and disease development.

This is particularly important in cancer research, because cancer is generally not caused by the alteration of a single gene or protein, but rather the dysregulation of entire signaling and regulatory networks. Mutations can lead to abnormal activation or inhibition of key biological components [1], modifying cellular decision-making processes and resulting in uncontrolled proliferation, apoptosis resistance and other cancerous mechanisms. Therefore, studying the pathways involved in disease progression and identifying the biological mechanisms affected by these alterations are essential for development of effective therapeutic strategies.

Due to the complex nature of these interactions in biological systems, we can analyse and model them with mathematical and computational models, such as Boolean networks. Boolean networks have become one of the most widely used methods for studying biological regulatory systems in the field of cancer research, as illustrated by the prevalent amounts of recent publications related to biological network and system modelling [2]*(Figure 1)*.

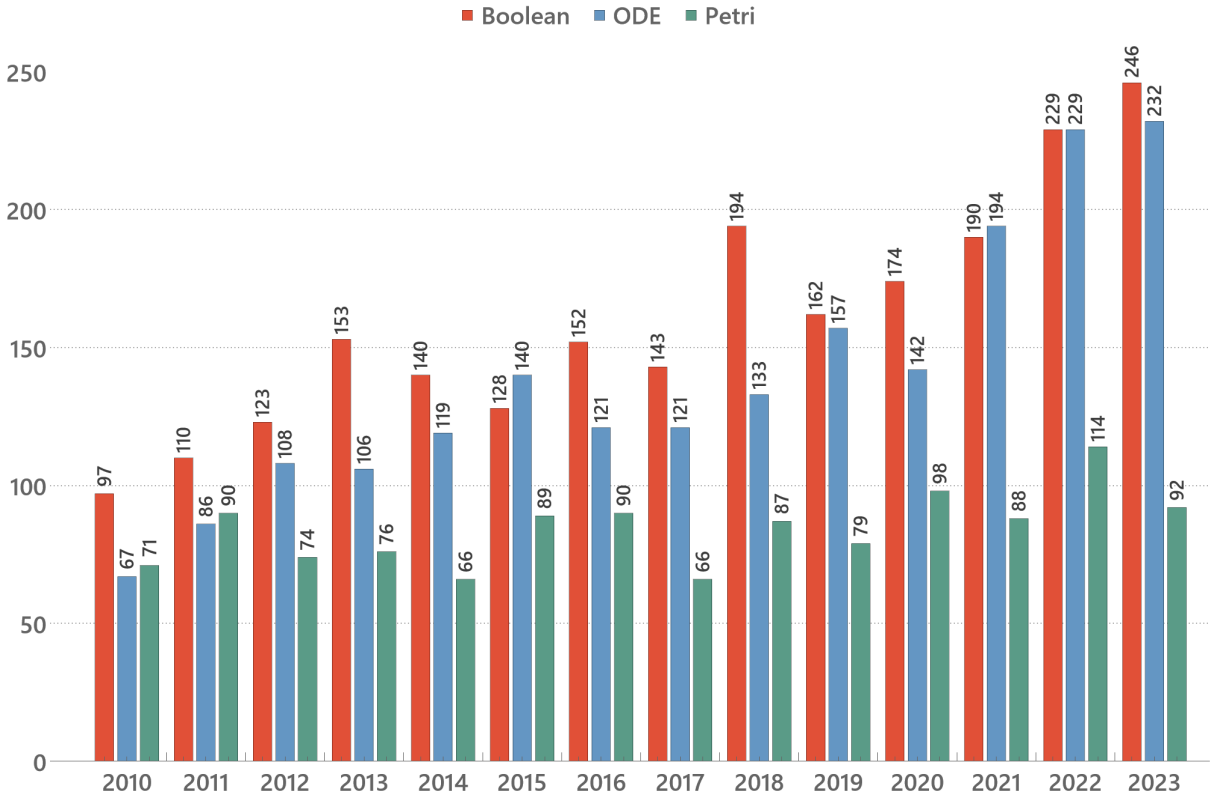


Figure 1: Evolution of numbers of publications on PUBMED through years according to different biological modelling frameworks, such as Boolean networks, Ordinary Differential Equations (ODE), and Petri nets. *The figure compares publication trends related to Boolean networks, ODE, and Petri nets in biological system modelling. The increasing number of publications associated with Boolean networks shows the importance of it in the research field, especially for the modelling of regulatory and signaling networks.*

A Boolean network is a discrete dynamical system and a mathematical model composed of a set of Boolean variables $X = \{x_1, x_2, \dots, x_n\}$ where $x_i \in \mathbb{B}$, representing biological entities such as genes or proteins [2]. Its evolution functions are defined as propositional logical formulas, $x'_1 = f_1(x), \dots, x'_n = f_n(x)$, with $x = (x_1, \dots, x_n)$. Each variable can take a binary value representing an active or inactive status. The evolution of the system is determined by logical rules that describe how the state of a component depends on the states of others within the network.

Boolean networks are widely used in biological modelling and are often considered as a gold standard for studying regulatory systems. Figure 1 presents the number of publications related to different modelling approaches clearly, highlighting the prominence of Boolean networks in systems biology. More particularly this model is easy to understand and particularly suitable because many biological mechanisms can be approximated by binary behaviours [5,6], allowing the study of large and complex systems.

One of the main advantages of Boolean modelling is its ability to describe large and complex regulatory systems without requiring detailed kinetic parameters or precise concentration measurements [5], which are often unavailable in biological studies. This allows researchers to model biological systems even when the knowledge is incomplete. As a result, Boolean networks can capture the qualitative behaviour of regulatory mechanisms and provide an informative representation of cellular decision-making processes.

Boolean network models have several applications in systems biology and cancer research. They can be used to simulate signalling pathways, study gene regulatory mechanisms, analyse cellular responses to perturbations and understand disease progression [4]. A key aspect of Boolean network analysis is the computation of attractors and stable states [5,7], which correspond to recurring patterns of activations, and which may be associated with distinct cellular phenotypes [3]. In cancer studies, the stable states represent different tumour subtypes, mutation profiles or drug-response behaviour. By simulating gene knockouts, protein inhibition or combinations of interactions [2,3,5], researchers can evaluate how perturbations affect the network dynamics and determine which pathological states can be considered. This contributes to identifying potential drug targets [4,10].

The scientific question addressed in this project is how Boolean network modelling can help identify the key actors in cancer, characterize the biological pathways involved and contribute to the discovery of potential drug targets through an accessible and interactive software platform.

Although Boolean networks provide powerful tools for studying biological systems, their adoption by biologists can be limited by the complexity of such software tools. Therefore, improving the usability and accessibility of the platform is important to enable its use by a broader scientific community.

Through an improved graphical interface of the software, the project aims to facilitate the modelling and manipulation of Boolean networks. Consequently, it allows researchers to explore cellular mechanisms more efficiently, analyse biological pathways involved in cancer development and identify variables that significantly influence pathological states [10].

B. Objective of internship

The main objective of our project was to improve and redesign the graphical user interface (GUI) of the platform BooN to make the design, visualization, and analysis of Boolean networks more intuitive and user-friendly. Therefore this task is considered as the core of Boolean network modelling.

To achieve this goal, we implemented several specific objectives, defined as:

- Developing an enhanced graphical interface that simplifies the construction and editing of Boolean networks, without depending on extensive help documentation.
- Implementing tools for graphical encoding of Boolean networks, which automatically convert user-defined interactions into formulas in Disjunctive Normal Form (DNF).
- Improving network specification by introducing more advanced interaction representations, enabling the modelling of more complex regulatory mechanisms.
- Preserving compatibility with the existing software architecture while extending its functionalities.

These enhancements were implemented within the existing BooN codebase to create a more interactive and user-friendly environment for Boolean network modelling. During this internship, the main objective was the design and implementation of new graphical interface components and functionalities within the BooN software.

Ultimately, the project's aim is to facilitate the exploration of biological regulatory systems and support researchers in studying signalling pathways, disease mechanisms, and potential therapeutic targets through an improved and user-friendly platform for Boolean network analysis.

II. Material & Methods

A. Description of Project

The internship project focused on the improvement of the graphical user interface of the BooN software platform. BooN is an open-source Python-based platform dedicated to the modelling and analysis of Boolean networks. It was developed by Pr. Franck Delaplace in February 2024 and is designed to support the study of genetic regulatory networks through Boolean network modelling.

The software provides several functionalities for Boolean network analysis, such as network construction, computation of stable states (equilibria), computation of model dynamics, and controllability analysis. BooN has been applied to biological case studies related to diseases, such as breast cancer [4,10], allowing researchers to study causal mechanisms and identify potential therapeutic targets.

The BooN project is organized into three main packages:

- *boon*: the core library which provides a set of methods for the manipulation and analysis of BooN objects.
- *logic*: a sub-library with basic functions related to propositional logic.
- *boonify*: the graphical user interface (GUI) developed using PyQt5.

BooN has an internal file format for the storage of networks (*.boon*) but also supports the import of models from other formats, such as BoolNet (Boolean Network Software Package), SBML (Systems Biology Markup Language) and Python-based formats.

For the graphical interface, the design of the Boolean network is done by the drawing of the interaction graph, from which the corresponding Boolean functions are automatically generated in Disjunctive Normal Form (DNF). The interface also provides different analysis, such as model dynamics simulations, controllability analysis and stable state computation *(Figure 2)*.

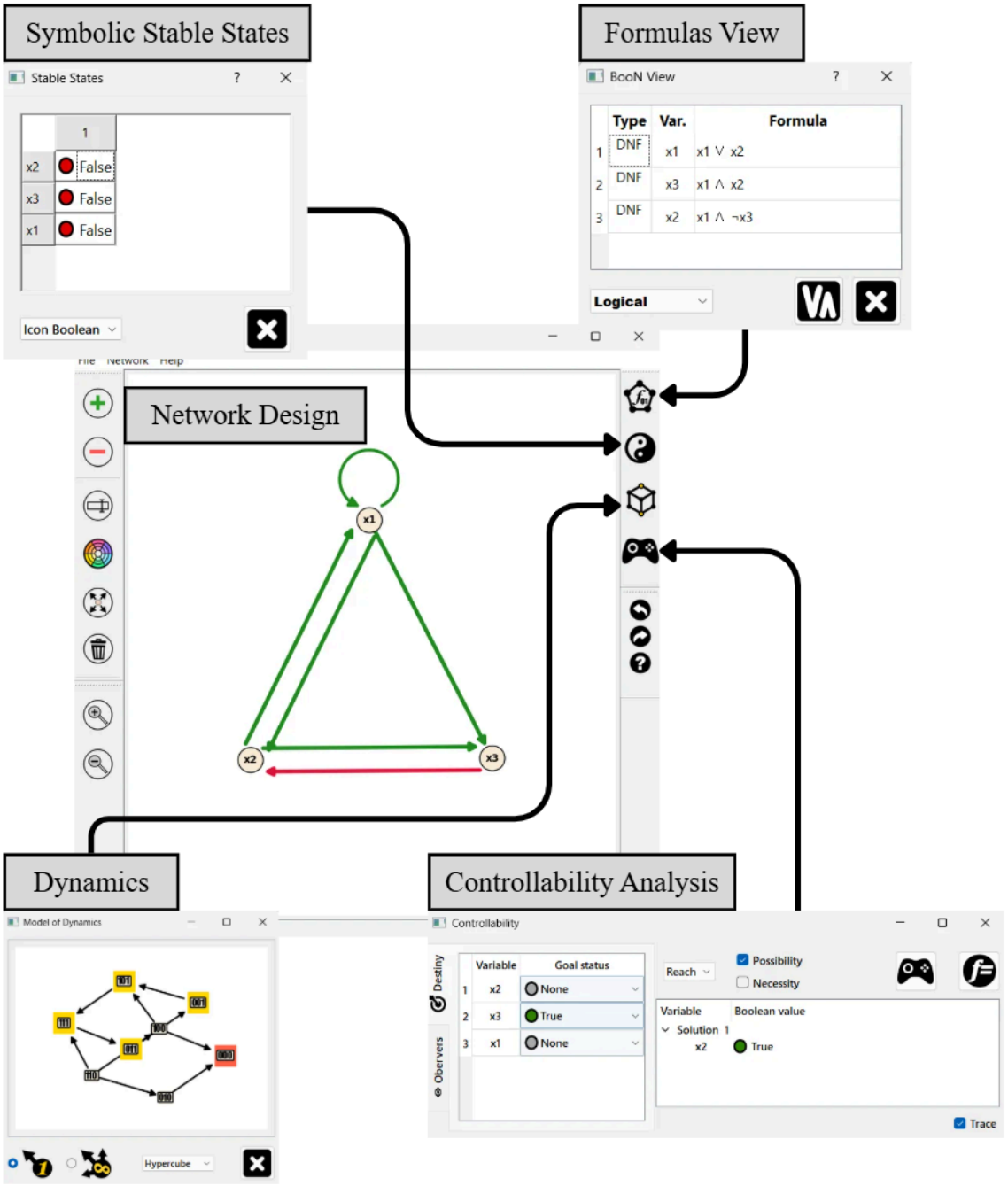


Figure 2: Overview of the BooN functionalities using GUI. *The design of the Boolean network is done from the drawing of the interaction graph. From which a DNF formula is then generated. We get the model's dynamics, referring to the transitions between states in both synchronous and asynchronous modes. The symbolic stable states computation, which refer to possible phenotypes. And finally, the controllability analysis to predict the variables that are controlled to reach a goal biological state.*

Despite the capabilities of the software, the existing version presented several limitations in terms of graphical creation and user manipulation. The construction of Boolean networks mainly required consulting documentation and understanding software-specific commands, making the software difficult to access for non-specialized users or researchers without prior experience. In addition, the graphical tools available for network editing were

limited in terms of specification of interactions, to represent more complex interactions and network structures.

To address these limitations, the internship therefore focused on enhancing the graphical user interface (GUI) of BooN to make the creation and analysis of Boolean networks more interactive and accessible to users. We wanted to implement graphical tools allowing users to create Boolean networks more intuitively, without help of documentation required. Users can construct interactive graphical network designs, which can then be converted into a DNF formula. We also wanted to add more graphical functionalities into the existing codebase to specify the interactions more in the network (edge family specification).

These improvements will provide a modernized version of the software capable of simplifying the graphical creation of Boolean networks for users, supporting advanced research applications, all while preserving the compatibility with the existing software architecture.

B. Methods and tools used

Programming and GUI framework:

The main programming language used in the project is Python. Python is widely used because of its readability, flexibility, object-oriented programming capabilities, and extensive collection of libraries for software development, data processing, and scientific computing. Its modular structure facilitates the development of large and complex applications while ensuring maintainability. The implementation of new graphical components and their integration into the BooN platform were carried out entirely in Python.

The graphical user interface is developed using the PyQt5 framework. PyQt5 is a Python library that provides bindings for the Qt framework, allowing creation of modern graphical interfaces. It includes a large collection of widgets, event-handling mechanisms, multimedia tools and graphical components that support the development of desktop applications.

Within the project, we used PyQt5 to manage the graphical representation of Boolean networks, including nodes, edges (interactions), logical operators and user interactions. It also enabled the implementation of advanced interface functionalities, such as node/edge selection, drag-and-drop operations, interactive network editing, and the customization of graphical elements (renaming/resizing nodes).

Qt Designer was used to simplify the interface development. This application is a graphical user interface design tool integrated into the Qt framework, that allows developers to create windows, menus, dialogs and control panels graphically before integrating them into the application. This tool generated *.ui* files, which is a special XML (eXtensible Markup Language) format, which can then be translated into a programming language such as Python or C++.

Computational methods:

In BooN, several computational analysis methods are integrated to support the modelling and analysis of Boolean networks.

Model Dynamics analysis is used to simulate and describe the evolution of the states of Boolean variables (genes or proteins) over time, based on logical rules defined within the network. This approach enables the study of biological system behaviour under different conditions and helps identify the effects of specific regulatory interactions *(Figure 3)*.

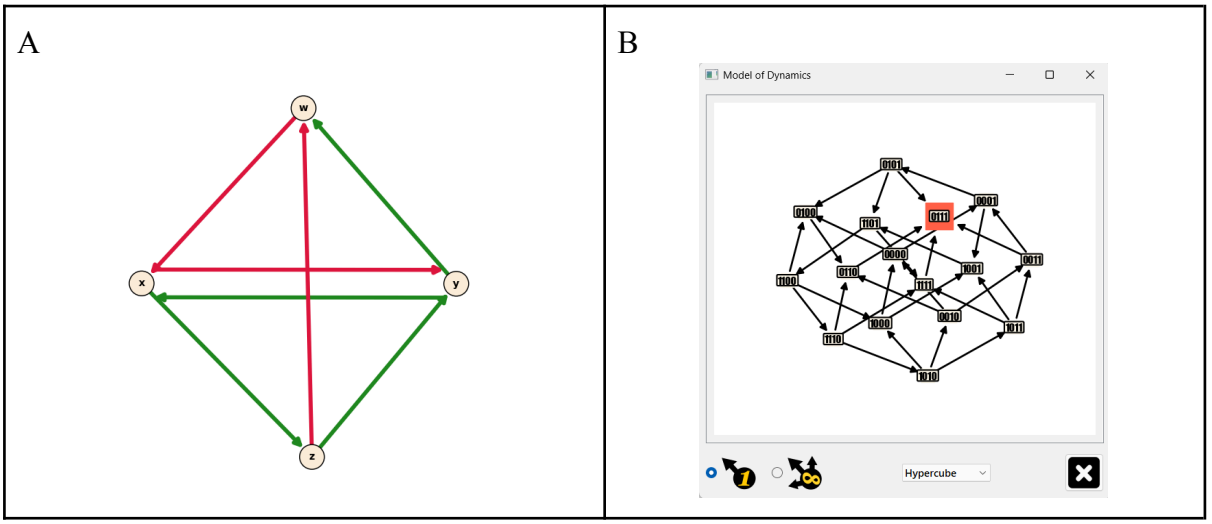
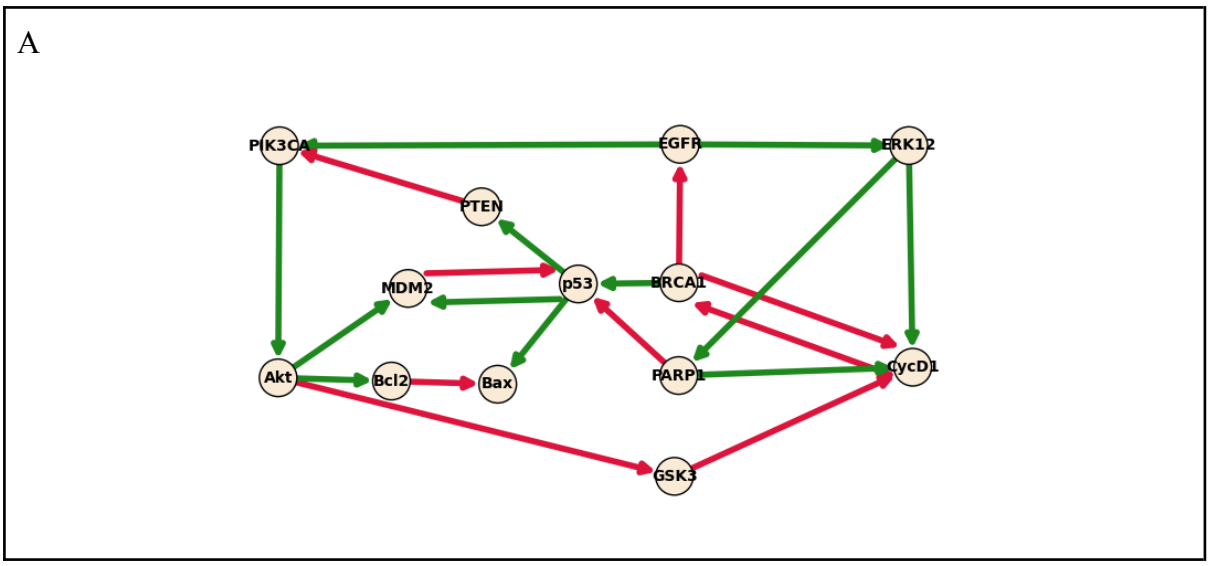


Figure 3: Interaction graph and its corresponding model's dynamics. ***(3.A)*** *This model is an example of a basic network with 4 nodes (genes/proteins), with interactions (activation/inhibition) between them.* ***(3.B)*** *The model of dynamics corresponding to the example model. It represents the evolution of states of all Boolean variables involved in the model (w, x, y, z).*

Two main paradigms can be used to model state transitions in Boolean networks: synchronous and asynchronous transitions. In synchronous models, each Boolean function is applied to compute a state transition from t to $t + 1$, if all components of the system require the same amount of time to change their value. In contrast, asynchronous models update components independently, allowing state change to occur at different times and capturing variations in biological processes. In the BooN platform, users can perform either synchronous or asynchronous simulations of their Boolean network *(Figure 3)*.

Stable State computation, which finds the equilibrium configurations in which the network no longer changes *(Figure 4)*. These stable states often correspond to biological cellular phenotypes (e.g., cell proliferation, apoptosis). The computation is based on SAT (Boolean Satisfiability) solvers, which are algorithms designed to efficiently solve Boolean satisfiability problems.



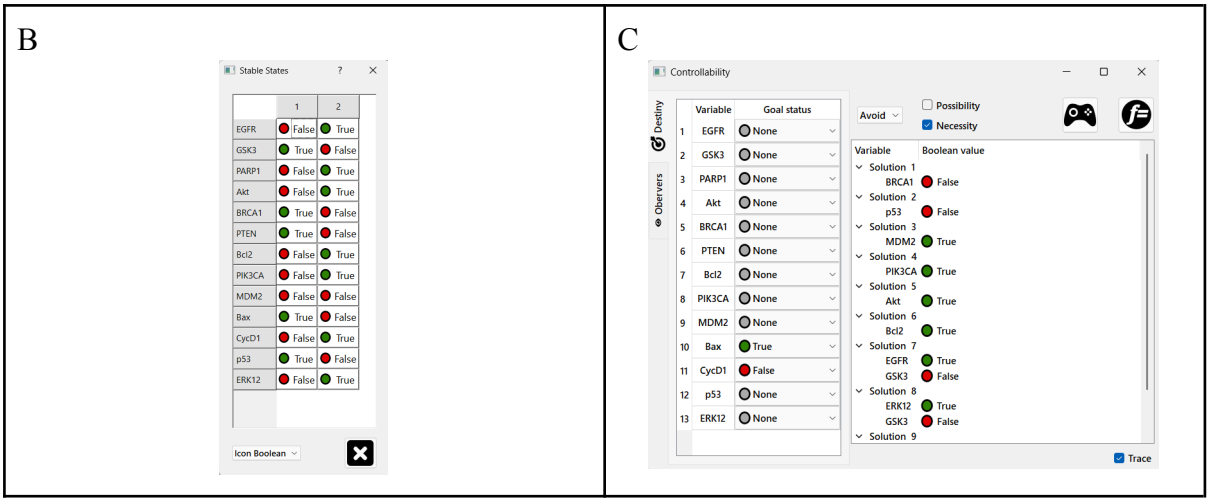


Figure 4: Interaction graph and stable states of case study of Breast cancer. ***(4.A)*** *The model of a regulatory network in normal cell conditions.* ***(4.B)*** *Stable states of the model. We note two possible stable states shown: corresponding to the apoptosis and the cellular division initiation.* ***(4.C)*** *Controllability analysis, which simulates cancerous condition by setting an apoptosis state, with the corresponding profile of the biomarkers "CycD1=False & Bax=True". We can program a query to get solutions describing the mutations required to trigger breast cancer/cancerous conditions.*

Controllability analysis aims to predict which variables should be controlled in order to reach a desired biological fate. This analysis is based on possibility and necessity queries, evaluating the influence of specific variables on the evolution of the network and their ability to drive the system toward a target configuration.

Together, these methods provide a complete framework for the creation, simulation, and analysis of Boolean networks.

Development methodology:

The implementation followed a modular software development approach, in which new graphical components were designed and implemented before being integrated into the existing BooN architecture. For this, the project used Git as a version control system, by enabling tracking of source code modifications, and allowing previous versions of the software to be restored. This methodology improved code organisation, simplified debugging and extension of the software. By developing components as independent modules, modifications were introduced into the application smoothly.

Design Choices:

The design of the new graphical user interface was guided by the principles of usability, clarity and consistency with the existing BooN interface. The main objective was to simplify the creation and editing of Boolean networks, while preserving the scientific rigor of the analysis provided by the software. The implemented features provided users with visual interactions, through actions such as node/edge selection, node resizing, graph customization, and network editing.

Software development process:

The redesign of the GUI was done on the file `boonify.py` to enhance the maintainability and extensibility of BooN. Therefore, as the number of functions has increased, the original code structure was restructured with new classes added, using an object-oriented architecture with related functionalities regrouped into classes. This new

structure separated interface management, graph editing, network conversion, analysis modules and background computations. This modular design improved the code readability, simplified debugging and facilitated integration of new functionalities while preserving compatibility with the existing BooN framework.

In addition, the original graphical implementation relied on an external library `(EditableGraph from netgraph)`, but this dependency was replaced by custom functions implemented in the newer version. This redesign gives better control of the graphical behaviour of the application and allows implementation of desired functionalities.

Testing and Validation procedures:

Functional testing was performed after incorporating the new components into the BooN platform to ensure compatibility with existing functionalities. These tests verified that the new graphical features interacted correctly with the network visualization tools and the pre-existing analysis modules of the software.

Validation was carried out using pre-existing examples of Boolean network models to confirm that the new implementations did not affect the correctness of the analytical results generated by BooN. The outputs produced before and after the modifications were compared to ensure consistency.

This validation process ensured that the new graphical implementations improved usability while preserving the scientific rigor and reliability of the platform.

III. Results obtained

We note that the improvements applied on the previous code added from 1434 to 3890 lines, with around 50 functions in the code to around 100 functions in the new version. We structured the code into several classes to each occupy a functionality of our software and simplify the readability of the code.

The code was structured with the following classes:

- `Boonify`: Main application window, file operations (open, save, import, export), history management (undo, redo), analysis launch (stable states, controllability analysis, etc);
- `Graph`: Interactive graph editor, node/edge manipulation (creation, deletion, renaming, resizing, family color assignment), mouse interactions (click, drag, double-click);
- `Network`: Conversion between the GUI graph and BooN model;
- `View`: Formula conversion of the Boolean network in multiple display styles (Logical, Python, Java, Mathematica, BoolNet);
- `StableStates`: Computation of stable states of the network;
- `Model`: Computation of model dynamics, for synchronous/asynchronous modes;
- `Controllability`: Control analysis of networks, to reach or avoid a target state, using possibility and necessity modalities;
- `Threader`: Background execution of long computations;
- `Help`: Documentation on the BooN platform;

The previous version of the BooN platform relied primarily on command-based interactions for network editing operations. Most actions required specific keyboard commands, making the tool difficult to master for users without extensive consultation of the documentation or prior experience with the platform.

To address these limitations, the graphical user interface was redesigned in the current version, through the introduction of a graphical toolbar that centralizes the main editing functionalities *(Figure 5)*.

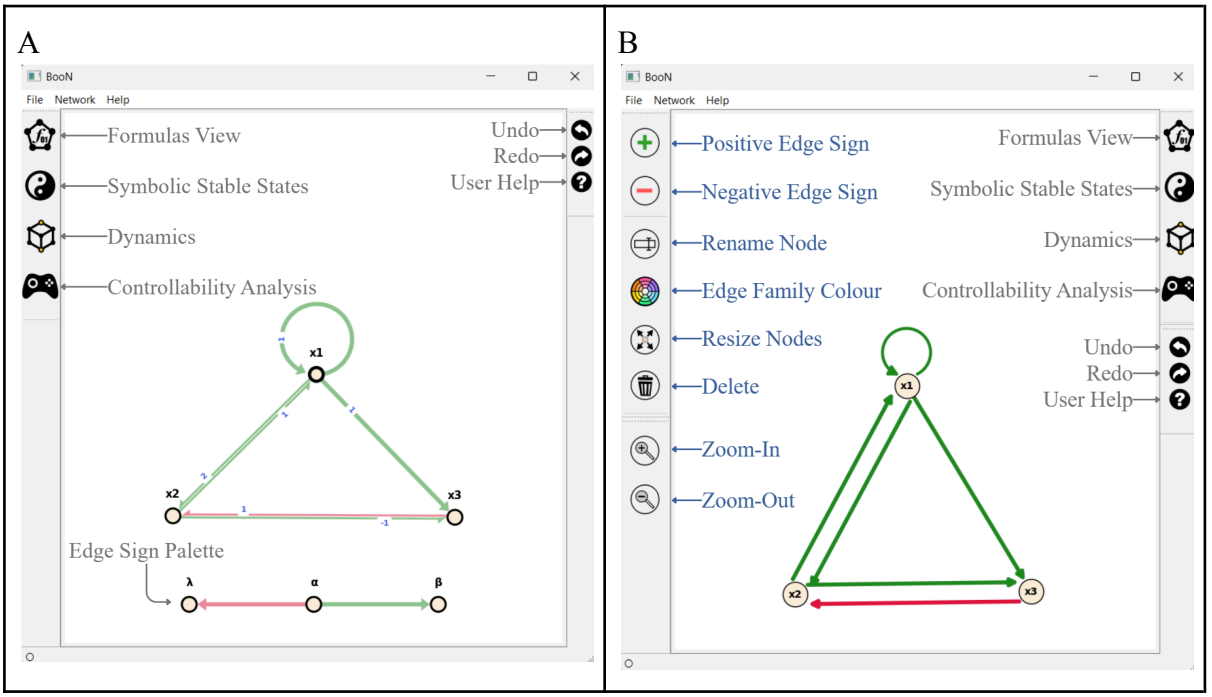


Figure 5: Comparison of the BooN interface between the previous (5.A) and current version (5.B). ***(5.A)*** *In the previous version, the node creation was controlled by key controls (press Shift and + key). The edge creation was with a double click on a starting node and release on a target node. The edge sign was specified by selecting the edge in the palette (λ, a, β) at the bottom of the canvas to assign the edge sign before creation.* ***(5.B)*** *In the current version, the node creation is done with a left click on the canvas, and the edge is created with a double click on the starting node and released on the target node. The edge sign can be specified in the toolbar at the left of the canvas, or even modified with a right click on the edge to shift the sign.*

The newly implemented toolbar provides the following editing features:

	: To assign a positive edge sign (+1: activation), and drawn in green.
	: To assign a negative edge sign (-1: inhibition), and drawn in red.
	: To rename a selected node.
	: To assign a family color to the edge, and which can also be hidden from the graph.
	: To resize all nodes or only the selected one. Can also reposition names at the center or above nodes.
	: To delete selected nodes or edges.

	: To zoom-in on the graph.
	: To zoom-out on the graph.

Edge sign assignment:

In previous versions, interaction/edge signs were selected through a palette-based selection (λ, α, β), making the process less intuitive. The updated interface introduces dedicated buttons that allow users to directly assign activation and inhibition signs to the interactions. Furthermore, we implemented a right-click action on an edge, to allow users to switch between positive or negative interactions. This redesign simplifies edge sign specification by reducing the number operations required, making it more accessible for users.

Node renaming:

Previously, newly created nodes had no default name and users were required to assign names with a specific sequence of commands (press `enter` + input name + press `enter` again to compute), in order to compute the Boolean network.

However, in the updated version, nodes are automatically given default names upon creation, ensuring that they can be stored correctly and integrated into the network structure and history system. Then, users can also choose to rename the node by clicking on the button or by double left-clicking on a node. Additionally, to preserve network consistency, duplicate node names are automatically checked and prevented.

Edge family color assignment:

A new feature was developed to associate interactions/edges with specific families through color coding. This functionality enables the representation of cooperative and alternative gene regulations within Boolean network models. Gene interactions belonging to the same family, meaning they interact cooperatively, can be visually grouped using a common family color. Then gene interactions that are regulated alternatively can be differentiated by another family color. We note that interactions without family assignment retain the default white color. For better visualization, family colors can also be temporarily hidden during network editing.

Node and label resizing:

Additional customization options were added to improve network visualization. Compared to the previous version of the software, where the nodes were all at a default size, and the names were placed above the nodes. In the new version, users can modify the size of an individual node or apply a common size to all nodes in the network.

Furthermore, node labels can also be displayed either at the center of nodes or above them. These improvements enhance the readability of large networks and allow users to adapt visualization to their preferences.

Deletion of nodes and edges:

A dedicated deletion button was added to the toolbar, complementing the existing keyboard shortcut (`del`). This enhancement provides a more intuitive mechanism for

removing network elements (nodes/edges) and improves accessibility for users unfamiliar with keyboard commands.

Graph zooming:

Compared to the previous version, explicit zoom-in and zoom-out controls were implemented to facilitate visualization of complex Boolean networks. These controls allow users to adjust the display scale according to the size and complexity of the Boolean network, improving both editing and visualization.

Overall, this redesigned interface significantly improves usability and accessibility of the BooN platform. The introduction of a graphical toolbar simplifies the Boolean network construction and editing, while providing a more intuitive environment. The addition of edge family assignments extends the modelling complexity, by enabling the representation of cooperative and alternative regulatory mechanisms. The customization and visualization options further enhanced the user experience and facilitated the construction of complex network models.

Graphical encoding of Boolean networks:

The major implementation in BooN concerns the graphical encoding of Boolean networks. In the previous version, node creation required keyboard shortcuts (press `shift` and +), while edge creation required double-click interactions.

These operations have been simplified in the redesigned system, through direct graphical manipulation features:

- Node creation: left-click on the canvas creates a node at the selected position.
- Edge creation: right-click operation on the node creates an interaction between source and target nodes.

Then, additional selection tools were implemented, such as:

- Single selection: left-click on node or edge to select it.
- Multiple selection: left-click on empty space and drag it to create the multiple selection rectangle; Another alternative is a keyboard shortcut, with the `ctrl` key pressed to select multiple nodes or edges desired.

Another feature was the dynamic self-loop angle adjustment mechanism, which was implemented to improve network readability. Self-regulatory interactions are automatically positioned to minimize overlap with other interactions, resulting in clearer network layouts.

The most significant functional extension introduced is the implementation of family colours for interactions and their integration into the generation of DNF formulas. This feature was added to enable the representation of cooperative and alternative gene regulation, and model more complex Boolean networks.

Family colors are used to distinguish interactions that participate in the same regulatory process. During the conversion of a network into DNF formulas, interactions sharing the same family can be combined to represent cooperative regulation, while interactions belonging to different families can model alternative regulatory pathways. Therefore, the generated logical formulas reflect more accurately the complexity of biological mechanisms.

This extension provides a richer modelling framework than the binary (activation/inhibition) representation and allows the study of regulatory systems involving cooperation and alternative mechanisms *(Figure 6)*.

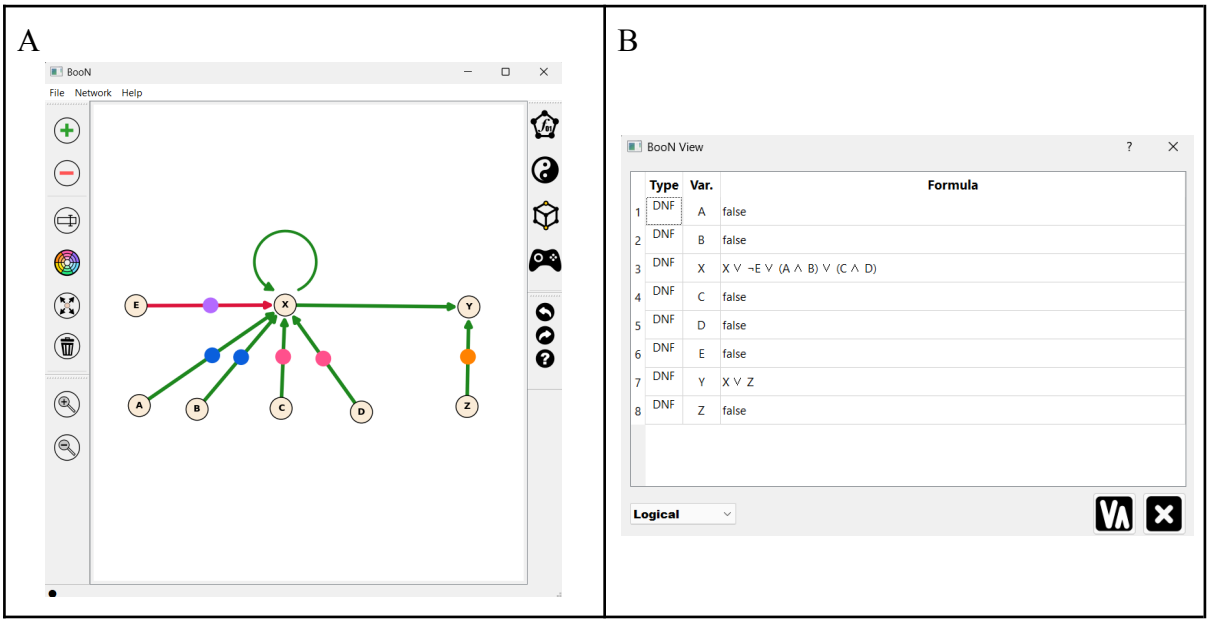


Figure 6: Demonstration of family color assignment on an example regulatory network and its conversion into the corresponding DNF formulas. ***(6.A)*** *In the example network model, we modeled inhibition/activation of genes, self-regulations, cooperative and alternative regulation. Here, we have a cooperative gene regulation of genes A & B of gene X (blue family), which is an alternative to the cooperative regulation of genes C & D of gene X (pink family). Gene E is also an alternative regulation of gene X (purple family), which also self-regulates (no color = white family). Finally, gene X (no color = white family) is an alternative regulation of gene Z (orange family) on gene Y.* ***(6.B)*** *After the Boolean network constructed, we get its conversion into DNF formulas, which is the following:* $(X = (A \wedge B) \vee (C \wedge D) \vee \neg E \vee X)$ *;* $(Y = X \vee Z)$*. These formulas reflect accurately the cooperative/alternative and other regulation specificities of the Boolean network modeled.*

Following the implementation of these new functionalities, validation procedures were conducted to verify their correctness and ensure compatibility with existing BooN models.

Software validation:

We extensively test the different functionalities based on several debugging and verification procedures. Dedicated print functions were used to monitor internal states and confirm the correct behaviour of the newly implemented features codes. These tests facilitated the identification and debugging of implementation issues.

A series of test Boolean networks were constructed to verify that all editing operations, logical computations, and DNF conversions remained functional. The tests confirmed that the new features integrated correctly with the existing architecture while preserving compatibility with previous functionalities.

Finally, after all these implementations were added into the BooN software, the validation results demonstrated that the redesigned interface, graphical encoding

mechanisms, and family-color-based interaction system operate correctly and extend the capabilities of the BooN platform without compromising existing behaviour.

The successful integration of these developments provides users with a more accessible and flexible environment for the construction, visualisation, and analysis of Boolean networks. In addition, the introduction of family-color-based interactions expands the modelling possibilities of the platform by enabling the representation of more complex regulatory mechanisms.

Overall, the obtained results demonstrate the successful implementation of the objectives defined for this work and highlight the added value of the proposed extensions to the BooN platform.

IV. Discussion & Perspectives

Contributions and strengths of the developed version of BooN:

The implementations integrated in this project have significantly improved both the accessibility and modelling functionalities of the BooN platform. The redesign of the graphical user interface has transformed the software from a command-based environment into a more intuitive graphical application [2]. With the implementation of a toolbar centralizing the editing functionalities, all operations are more accessible to users.

The introduction of direct graphical manipulation mechanisms have simplified the construction of Boolean networks. Node and edge creation, selection tools, and visualization controls provide a more natural workflow compared to the previous command-based version [2].

A major contribution of this work is the implementation of family colors for interactions, which allows the representation of cooperative and alternative regulatory mechanisms. These improvements enhance the user-experience and facilitate the creation of more complex network models.

Limitations of the implementation:

Furthermore, some limitations are inherent to Boolean network modelling itself. Boolean networks represent biological entities using binary states [6], typically active or inactive, which simplifies real biological systems modelling. Gene expression and protein activity often vary continuously and may depend on concentration levels. Consequently, certain subtle regulatory effects cannot be captured by purely binary Boolean models.

Another limitation concerns the absence of an explicit temporal dimension, we could expand the Boolean model to be time-continuous [6]. Standard Boolean networks describe state transitions but do not directly represent reaction rates or timing differences between biological processes [6]. As a result, biological events that occur at different speeds may be represented identically, which is why Boolean networks are considered as first-level logical representation of biological systems rather than fully predictive models [5,6]. Nevertheless, they remain a valuable tool for studying regulatory mechanisms, generating hypotheses and identifying key interactions within complex biological networks.

Perspectives:

For future developments of the platform, an improvement could be the integration of biological pathway databases, allowing users to import existing regulatory networks directly into the software. Such functionality can accelerate model construction and reduce manual effort required to create large-scale networks.

To study tissue level dynamics, in cancer context, another approach could be based on cellular automata, allowing representation of spatial propagation effects and diffusion-like behaviours within biological systems.

Also the integration of artificial intelligence techniques could further enhance the platform by supporting users in different tasks [9], such as suggesting potential missing interactions based on existing network structures data [1], suggesting node naming, and detecting possible inconsistencies in networks.

Finally, some user-oriented improvements could include automatic layout optimisation for large networks and grouping between interaction families.

These enhancements could make BooN a more adaptive and user-friendly environment for Boolean network modelling, in order to appeal to a broader scientific field.

V. Conclusion

This internship's main objective was to redesign and enhance the graphical user interface of BooN platform, in order to make the construction, visualization and analysis of Boolean networks more accessible to users.

The graphical interface was redesigned through the implementation of a toolbar centralizing editing functionalities and graphical interaction mechanism which simplified editing operations. In addition, new functionalities were introduced to improve network representation, such as enhanced node and edge manipulation, visualisation tools and implementation of family-colour-based interactions. The implementation of family colouring into the generation of DNF formulas, allows the representation of more complex Boolean network modelling, including cooperative and alternative regulatory mechanisms.

These improvements contributed to make Boolean network creation and manipulation more accessible to a broader field of users, while preserving analytical functionalities of the platform. The new functionalities had been validated to operate correctly and remained compatible with the existing BooN architecture, ensuring reliability of the software.

Finally, this opens several perspectives for future developments, including the integration of biological databases, the extension of modelling approaches toward spatial or temporal frameworks [6], and the incorporation of artificial intelligence tools to further support users in network construction and analysis [9]. These directions could potentially further enhance BooN platform as a powerful and flexible environment for the study of biological regulatory systems.

VI. Bibliography

[1] : Sontag, Eduardo. (2005). *Robustness and fragility of Boolean models for genetic regulatory networks.* **Journal of Theoretical Biology.** https://doi.org/10.1016/J.JTBI.2005.01.023

[2] : Delaplace, F. (2025). *BooN: Boolean Network Analysis Software.* **Bioinformatics Advances**, 5(1). Oxford University Press. https://doi.org/10.1093/bioadv/vbaf082

[3] : Paulevé, L. et al. (2024). *Boolean Network Generator for Control and Reduction.* **Peer Community In**, 4(). https://doi.org/10.48550/arXiv.2406.02304

[4] : Delaplace, F. (2024). *Case Study: Breast Cancer Cause and Drug Target Inference.* **GitHub Repository**. Available at: https://github.com/Franck-Delaplace/BooN/wiki/Case-Study:BreastCancer

[5] : Schwab, J. D., Kühlwein, S. D., Ikonomi, N., Kühl, M., & Kestler, H. A. (2020). *Concepts in Boolean network modelling: What do they all mean?* **Computational and Structural Biotechnology Journal**, 18, 571–582. https://doi.org/10.1016/j.csbj.2020.03.001

[6] : Bornholdt, S. (2008). *Boolean network models of cellular regulation: prospects and limitations.* **Journal of the Royal Society**, Interface, 5 Suppl 1(Suppl 1), S85–S94. https://doi.org/10.1098/rsif.2008.0132.focus

[7] : Albert, I., Thakar, J., Li, S. *et al.* (2008). *Boolean network simulations for life scientists.* **Source Code Biol Med 3**, 16. https://doi.org/10.1186/1751-0473-3-16

[8] : Sick, B. (2024). *boolean.py: A Python Library for Boolean Algebra*. **GitHub Repository**. Available at: https://github.com/bastikr/boolean.py

[9] : Lin, F. (2025, July 10). *Chan Zuckerberg Initiative's AI model captures gene networks to address disease.* **Genetic Engineering & Biotechnology News.** https://www.genengnews.com/topics/artificial-intelligence/chan-zuckerberg-initiatives-ai-model-captures-gene-networks-to-address-disease/

[10] : Biane, C., & Delaplace, F. (2019). *Causal reasoning on Boolean control networks based on abduction: Theory and application to cancer drug discovery.* **IEEE/ACM Transactions on Computational Biology and Bioinformatics**, *16*(5), 1574–1585. https://doi.org/10.1109/TCBB.2018.2889102

VII. Appendices

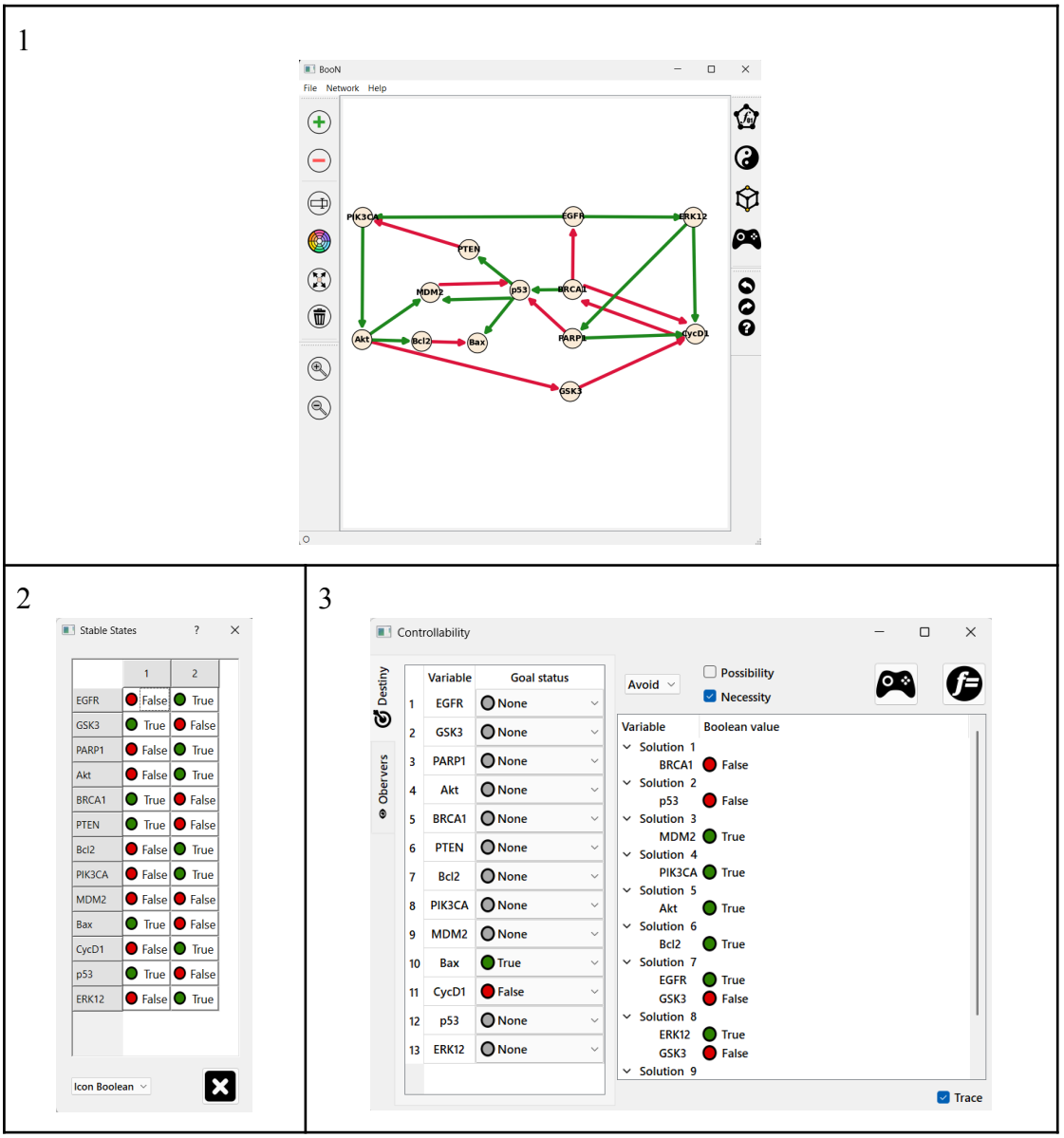
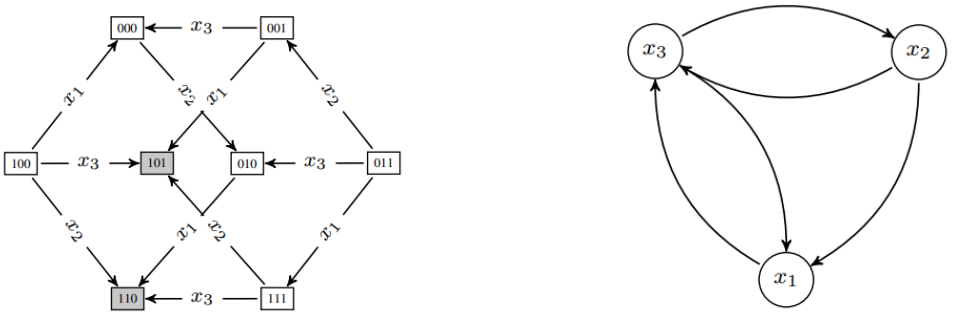


Figure A: Case study of breast cancer with BooN. ***(A.1)*** *The model depicts the regulatory network in normal cell conditions.* ***(A.2)*** *Computation of stable states of the model. We note two possible stable states shown, which respectively correspond to the apoptosis and the cellular division initiation. A cancerous situation is characterized here by a particular hallmark of cancer: the loss of apoptosis, enabling cell immortality. Cell division thus becomes the only phenotype identifying the development of metastases. The biomarkers are CyC1 and Bax and their state distinguishes the phenotypes. The following profile of both biomarkers at a stable state corresponds to apoptosis: CycD1=False, Bax=True. By contrast, when CycD1=True, Bax=False the stable state represents the cell division. These two bio-marking exist at stable states in the initial normal cell condition.* ***(A.3)*** *Controllability analysis. The cancerous conditions set Cyc1 to False and Bax to True, with [Avoid] and [Necessity] selected. After execution of the query, 3469 Models are produced and 9 solutions are found. The solutions describe the mutations required to trigger breast cancer.*



$$F = \{x_1 = x_2 \vee x_3, x_2 = \neg x_3, x_3 = \neg x_2 \wedge x_1\}$$

Figure B: Model of asynchronous dynamics and its corresponding interaction graph of function F defined. *The model of dynamics describes all the trajectories of the model. The interaction graph conveys the directed interactions (activation/inhibition) between variables of a Boolean network (gene/protein).*

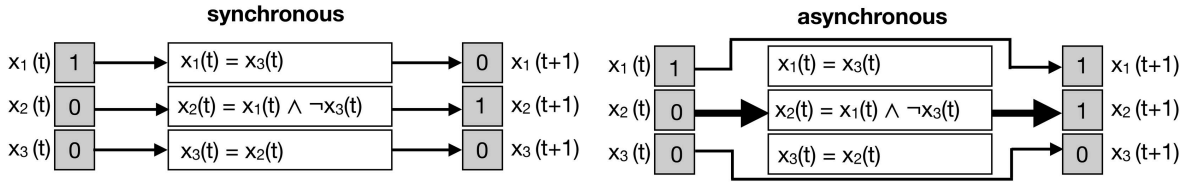


Figure C: Paradigms of state transition in Boolean network models. *We note the two following paradigms of state transition from state x(t) to x(t+1). In synchronous models, all functions are applied at the same time, while asynchronous models have only one random function (bold arrow) updated per step.*

Class `Boonify`:

```
def __init__(self):
def on_graph_changed(self):
def setup_callbacks(self):
def open(self):
def save(self):
def saveas(self):
def importation(self):
def exportation(self):
def quit(self):
def closeEvent(self, event):
def history_raz(self):
def undo(self):
def redo(self):
def add_history(self):
def    add_color_history(self,
boon_snapshot=None):
def show_history(self):
def refresh(self):
def help(self):
def view(self):
def stablestates(self):
def model(self):
def controllability(self):
def   display_saved_flag(self,
val: bool = True):
```

Class `Graph`:

```
def __init__(self, canvas):
def setup_design(self, boon):
def
_track_node_positions(self,
dx, dy):
def
_commit_tracked_positions(self
):
def    _apply_edge_color(self,
rgb):
def     change_edge_sign(self,
sign):
def
set_default_edge_sign(self,
sign):
def _add_new_node(self, x, y):
def
refresh_next_node_id(self):
def next_default_label(self):
def _create_edge(self, source,
target):
def
_compute_self_loop_angle(self,
node):
def _draw_nodes(self):
def _draw_labels(self):
def _draw_edges(self):
```

```
def   _handle_double_click(self,
node):
def   _select_node(self,   node,
shift_pressed=False):
def
_select_nodes_in_rectangle(self,
x1, y1, x2, y2):
def delete_selection(self):
def
_delete_selected_nodes(self):
def _delete_selected_edge(self):
def  _get_nearest_node(self,  x,
y, threshold=0.05):
def  _get_nearest_edge(self,  x,
y, threshold=0.045):
def _canonical_edge(self, u, v):
def _get_event_xy(self, event):
def   _move_selected_nodes(self,
dx, dy):
def rename_node(self):
def
_point_to_segment_distance(self,
px, py, x1, y1, x2, y2):
def zoom_in(self):
def zoom_out(self):
def reset_zoom(self):
def pick_edge_color(self):
```

Class `Network`:		

`def __init__(self):`
`def graph_to_boon(self, graph_editor, current_boon=None):`
`def boon_to_graph(self, boon, graph_editor):`
	`def _draw_normal_edge(self, u, v):`
	`def _draw_bidirectional_edges(self, u, v):`
	`def _draw_self_loops(self):`
	`def _draw_family_circles(self):`
	`def _apply_zoom(self):`
	`def _draw_edge_preview(self, x1, y1, x2, y2):`
	`def _clear_edge_preview(self):`
	`def toggle_family_colors(self):`
	`def toggle_edge_sign(self, edge):`
	`def redraw_graph(self):`
	`def on_canvas_press(self, event):`
	`def on_canvas_release(self, event):`
	`def on_canvas_motion(self, event):`
	`def on_key_press(self, event):`
		`def set_edge_color_from_palette(self, color_name):`
		`def setup_color_menu(self, parent):`
		`def open_color_palette(self):`
		`def build_color_palette(self):`
		`def show_more_colors(self):`
		`def show_less_colors(self):`
		`def setup_resize_menu(self, parent):`
		`def open_resize_menu(self):`
		`def build_resize_palette(self):`
		`def _resize_increase(self):`
		`def _resize_decrease(self):`
		`def _sync_node_sizes_to_boon(self):`
		`def set_family_color(self, color_value):`
		`def _wrap_node_label(self, label, max_length=7):`

Class `View`:		

`def __init__(self, parent=None):`
`def initialize_view(self):`
`def change_formula(self):`
`def cb_styling(self):`
`def convertdnf(self):`		

Class `StableStates`:	Class `Help`:	

`def __init__(self, parent=None):`
`def cb_styling(self):`
`def stablestates(self):`
	`def __init__(self, parent=None):`	

Figure D: Table of all functions implemented in each class in the main graphical user design code "boonify.py". *Each class works on a part of the code, such as* `Boonify` to manage the main application window, file operations, analysis and user actions; `Graph` provides interactive graph editing and its visualization; `Network` converts between the GUI graph representation and BooN model; `View` displays Boolean formula in multiple formats (Logical, Python, Java, Mathematica, BoolNet); `StableStates` computes the stable states of the network; `Model` simulates the model dynamics in synchronous and asynchronous update modes; `Controllability` does control analysis to determine how target states can be reached or avoided; `Threader` executes intensive computation tasks in the background; `Help` provides a guide-documentation on the BooN platform to users;

Github repository of code source: https://github.com/s-boomika/Stage_BOON